

Cụm 5 - Fundamental Architecture Styles

Mục lục

5.1 Tổng quan Cụm 5: Fundamental Architecture Styles	3
Style vs Pattern	3
9 Architecture Styles trong khoá	4
Cụm 5 (Fundamental - monolithic-friendly)	4
Cụm 6 (Distributed)	4
Bài trong cụm	4
5.2: Monolithic vs Distributed	4
5.3: Layered Architecture	4
5.4: Pipeline Architecture	4
5.5: Microkernel Architecture	5
Tại sao chỉ 3 style + Monolithic baseline ở Cụm 5?	5
Cách đọc cụm	5
5.2 Monolithic vs Distributed	6
Định nghĩa	6
Monolithic Architecture	6
Distributed Architecture	6
Monolithic là default	6
Lợi ích monolithic	8
Khi nào monolithic không đủ	8
8 Distributed Computing Fallacies	8
Hidden costs của Distributed	8
Cost 1: Latency và bandwidth	8
Cost 2: Partial failure	8
Cost 3: Eventual consistency	9
Cost 4: Distributed tracing	9
Cost 5: Operational complexity	9
Cost 6: Testing	9
Cost 7: Network security	9
Trade-off table	9
Modular Monolith — middle ground	10
Strangler Fig pattern	10
Sai lầm thường gặp	10
Sai lầm 1: Microservices vì hype	10
Sai lầm 2: Distributed monolith	10
Sai lầm 3: Quên 8 fallacies	10
Sai lầm 4: Không có observability từ ngày 1	10

Sai lầm 5: Premature optimization	10
Tóm tắt	11
5.3 Layered Architecture	12
Topology	12
Closed vs Open layers	12
Closed layer (default)	12
Open layer	13
Sinkhole Anti-pattern	13
Khi dùng Layered	14
Trade-off với QA	14
Implementation trong code	14
Java/Spring example	14
Python/Flask example	15
Variants	15
Layered + Hexagonal	15
Onion / Clean Architecture	15
Sai lầm thường gặp	16
Sai lầm 1: Skip layers ad-hoc	16
Sai lầm 2: Layered cho complex domain	16
Sai lầm 3: Database-first design	16
Sai lầm 4: Repository pattern abuse	16
Tóm tắt	16
5.4 Pipeline Architecture	17
Topology cốt lõi	17
Bốn loại filter	17
1. Producer (source)	17
2. Transformer	17
3. Tester	17
4. Consumer (sink)	17
Unix Philosophy	17
Khi dùng Pipeline	18
Cách implement	18
Synchronous (in-process)	18
Async với queue (in-process)	18
Distributed (network)	19
Trade-off với QA	19
Ví dụ thực tế	19
Compiler (GCC, LLVM)	19
ETL Pipeline (Airflow, Luigi)	20
Image Processing (PIL, OpenCV)	20
CI/CD Pipeline	20
Variants	20
Linear pipeline	20
Branching pipeline	20
DAG pipeline (Directed Acyclic Graph)	21
Sai lầm thường gặp	21
Sai lầm 1: Stateful filter	21

Sai lầm 2: Tightly coupled filters	21
Sai lầm 3: God filter	21
Sai lầm 4: No error handling	21
Sai lầm 5: Pipeline cho non-pipeline workload	21
Tóm tắt	22

5.5 Microkernel Architecture 23

Topology cốt lõi	23
Lợi ích vs Layered	23
Examples thực tế	24
VS Code	24
Eclipse IDE	24
Chrome / Firefox	24
WordPress	24
CMS / E-commerce platforms	24
DAW (Digital Audio Workstation)	24
Plug-in Contract Design	24
Loại 1: Event/Hook system	24
Loại 2: Service registration	25
Loại 3: Command pattern	25
Loại 4: Contribution points (Eclipse model)	25
Plug-in Lifecycle	26
Sandboxing (cho third-party plug-ins)	26
Process isolation	26
Sandboxed runtime	26
Permission system	26
Trade-off với QA	26
Khi dùng Microkernel	27
Variants	27
Microkernel + Distributed plug-ins	27
Microkernel + Module Federation (Frontend)	27
Sai lầm thường gặp	27
Sai lầm 1: Core quá lớn	27
Sai lầm 2: API không backward compatible	27
Sai lầm 3: Plug-in coupling lẫn nhau	28
Sai lầm 4: Performance overhead bị bỏ qua	28
Sai lầm 5: Security model yếu	28
Tóm tắt	28
Tổng kết Cụm 5	28

5.1 Tổng quan Cụm 5: Fundamental Architecture Styles

Tóm tắt một dòng: Architecture Style là “shape” tổng thể của hệ. Cụm này dạy 4 style fundamental (monolithic-friendly): Monolithic baseline, Layered, Pipeline, Microkernel — và quyết định cơ bản nhất: monolithic hay distributed.

Style vs Pattern

Phân biệt:

- **Architecture Style**: shape tổng thể của hệ. Vd: “monolithic”, “microservices”, “event-driven”. Quyết định một lần cho cả hệ (thường).
- **Architecture Pattern**: solution cụ thể cho một bài toán con. Vd: “Saga”, “Circuit Breaker”, “CQRS”, “Strangler Fig”. Một hệ có thể dùng nhiều pattern.

Trong khoá này, Cụm 5 + 6 đi qua 9 *styles*. Pattern được nhắc lúc relevant nhưng không là focus chính.

9 Architecture Styles trong khoá

Cụm 5 (Fundamental - monolithic-friendly)

Style	Topology	Khi dùng
Layered	Tầng UI/Business/Data	Default cho monolith, app đơn giản
Pipeline	Filter nối tiếp	Data processing, ETL, compiler
Microkernel	Core + plug-ins	IDE, browser, CMS, product có extensibility

Plus Monolithic vs Distributed comparison (Bài 5.2).

Cụm 6 (Distributed)

Style	Topology	Khi dùng
Service-based	4-12 coarse services + DB shared	Enterprise medium, ít risk hơn microservices
Microservices	Many fine-grained services + DB per service	Large scale, multi-team
Event-Driven	Async message broker	Real-time, high throughput, decoupling
Space-Based	In-memory grid + DB async sync	Extreme load (Black Friday)

Bài trong cụm

5.2: Monolithic vs Distributed

Trade-off cơ bản nhất. Vì sao monolithic là *default*, khi nào lên distributed. “Distributed Computing Fallacies” — 8 sai lầm phổ biến. Cost của distributed: latency, partial failure, debugging.

5.3: Layered Architecture

Style phổ biến nhất historical. UI / Business / Data layer. Closed vs Open layer. Strengths: simple, well-understood. Weaknesses: hard to scale parts, “sinkhole anti-pattern”.

5.4: Pipeline Architecture

Filter + Pipe. Sequential transformation. Unix philosophy. Strengths: composable, easy to add filter. Use cases: ETL, compiler, image processing.

5.5: Microkernel Architecture

Core (kernel) + Plug-ins. OCP at scale. Examples: VS Code, Eclipse, Chrome, WordPress. Strengths: extensibility, third-party ecosystem. Weaknesses: plugin compatibility, performance overhead.

Tại sao chỉ 3 style + Monolithic baseline ở Cụm 5?

Cụm 5 cover **fundamental styles** — những style mà toàn hệ chạy trong 1 process (typical monolith), hoặc 1 process với plug-ins. Cụm 6 cover **distributed styles** — nhiều process/network.

Reason tách: monolithic thinking và distributed thinking khác nhau cơ bản. Distributed thêm cost mới (network, partial failure, eventual consistency) cần cách reasoning mới.

Cách đọc cụm

Tuần tự 5.2 □ 5.3 □ 5.4 □ 5.5. Hoặc đọc 5.2 first và skip nếu đã biết, sau đó chọn style đang relevant với dự án mình.

Mỗi bài có pattern chung:

1. **Topology**: vẽ diagram cốt lõi.
2. **Khi nào dùng**.
3. **Khi nào không**.
4. **Trade-off** với QA (Cụm 4): style này tối ưu QA gì, hy sinh QA gì.
5. **Real-world examples**.

Vào [Bài 5.2: Monolithic vs Distributed](#) để bắt đầu.

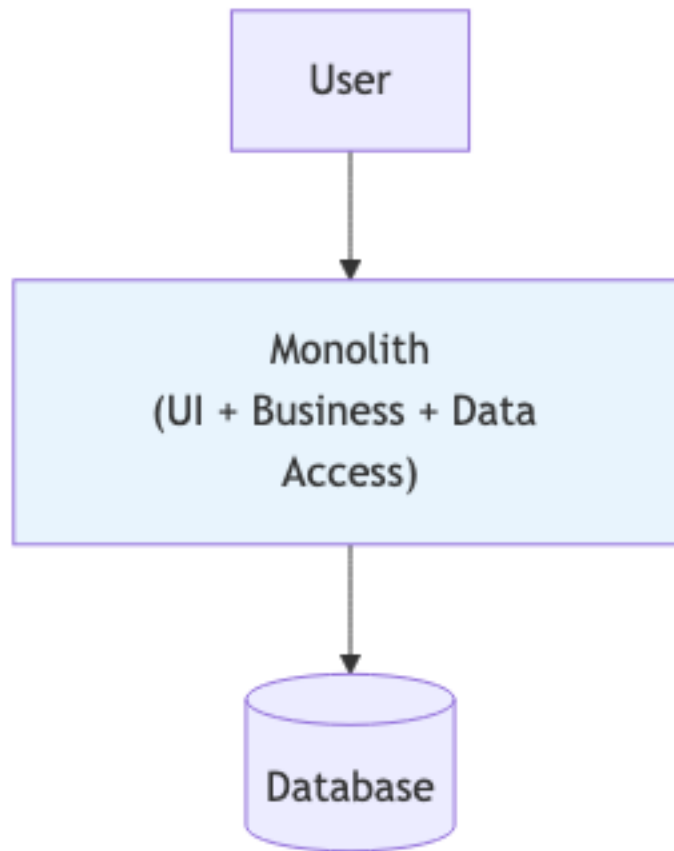
5.2 Monolithic vs Distributed

Tóm tắt một dòng: Monolithic = toàn hệ chạy 1 process; Distributed = nhiều process qua network. Monolithic luôn nên là default. Distributed *chỉ* khi có lý do mạnh — vì nó thêm 5x complexity và 5x operational cost.

Định nghĩa

Monolithic Architecture

Toàn bộ application code chạy trong 1 process duy nhất (hoặc 1 container/server, replicate cho HA).



Hình 1: Sơ đồ 10

Communication giữa các module = function call trong process. Fast (nanoseconds), reliable (no network).

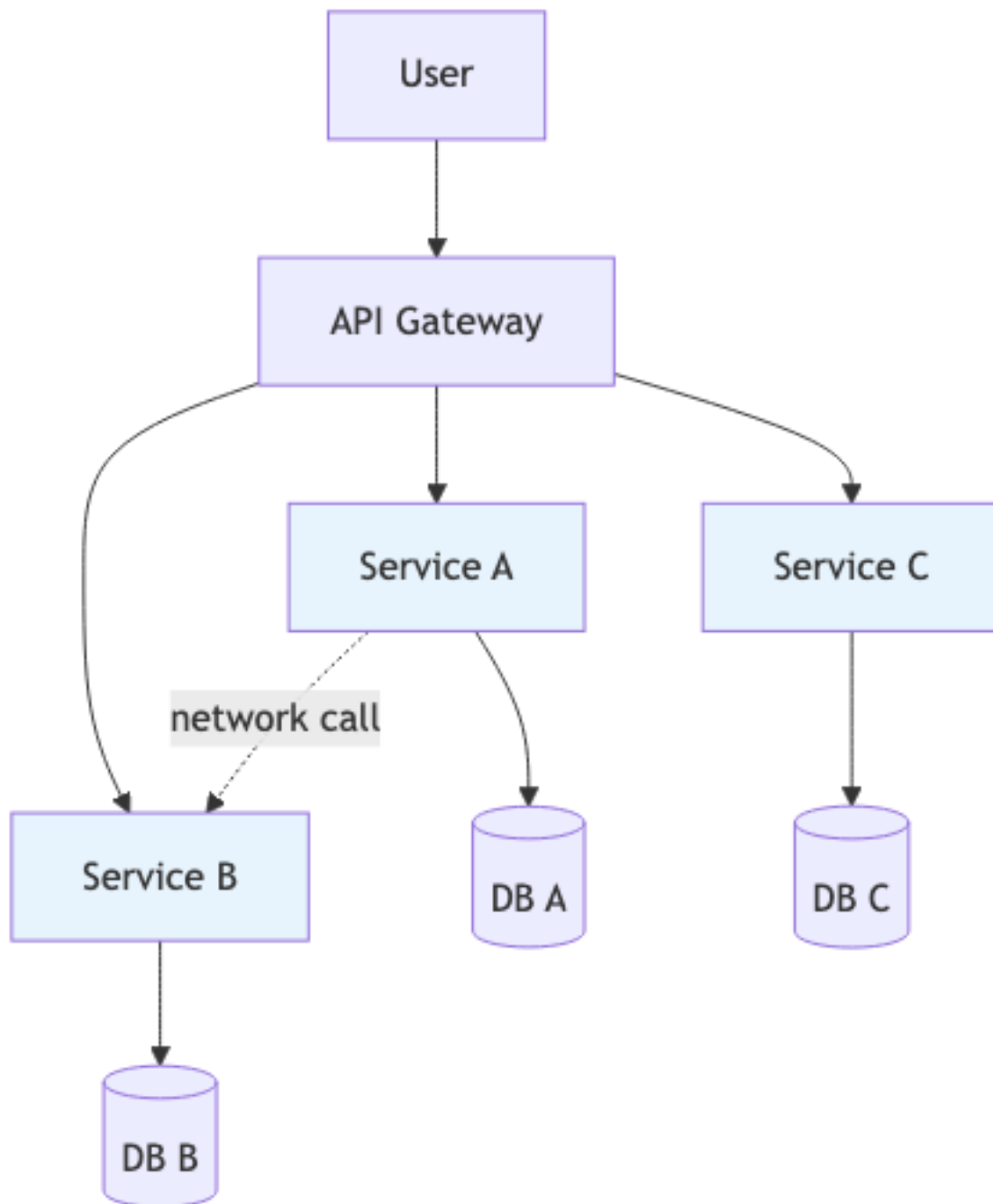
Distributed Architecture

Application chia thành nhiều process độc lập, giao tiếp qua network.

Communication = HTTP/gRPC/message. Slow (milliseconds), unreliable (network failures).

Monolithic là default

Quy tắc thực hành: nếu bạn chưa có lý do *cực mạnh* để distributed, hãy monolithic.



Hình 2: Sơ đồ 11

Lợi ích monolithic

1. **Đơn giản phát triển.** 1 codebase, 1 deploy, 1 database. Onboard dev nhanh.
2. **Performance cao.** Function call < 1 microsecond. Distributed call > 1ms.
3. **Transaction dễ.** ACID trong 1 database. Distributed transaction (saga, 2PC) phức tạp.
4. **Debug dễ.** Stack trace đầy đủ. Distributed: distributed tracing required (Jaeger, Zipkin).
5. **Cost thấp.** 1 server + 1 DB = \$50/tháng. Distributed setup minimum = \$500/tháng.

Khi nào monolithic không đủ

Tín hiệu phải lên distributed:

- **Team scale:** > 50 engineers, tuần nào cũng merge conflict trên monolith.
- **Different scaling:** 1 module ăn 90% traffic, không thể scale riêng.
- **Different tech stack:** ML team cần Python, web team cần Node, blockchain cần Rust.
- **Independent deploy:** cần ship 10 lần/ngày từ team này mà không block team kia.
- **Fault isolation:** 1 module down không được vỡ toàn hệ.

Nếu *không* match ≥ 2 điểm trên $\square$ monolithic vẫn ổn.

8 Distributed Computing Fallacies

Peter Deutsch + James Gosling (Sun Microsystems, 1994) liệt kê 8 sai lầm developer mới hay tin về distributed system:

1. **The network is reliable** — Sai. Packet drops, switches fail, cable bị đào lên.
2. **Latency is zero** — Sai. Network call 1-100ms vs local call < 1us.
3. **Bandwidth is infinite** — Sai. Có limit, có cost.
4. **The network is secure** — Sai. MITM, sniffing nếu không TLS.
5. **Topology doesn't change** — Sai. Nodes lên xuống, IPs thay đổi.
6. **There is one administrator** — Sai. Multiple teams config, có conflict.
7. **Transport cost is zero** — Sai. Network cost cao, đặc biệt cross-region.
8. **The network is homogeneous** — Sai. Heterogeneous protocol, latency, OS.

Mỗi fallacy = một class bug trong production. Distributed system architect phải handle cả 8.

Hidden costs của Distributed

Khi quyết định distributed, ngoài lợi ích visible, có cost ẩn:

Cost 1: Latency và bandwidth

Local call = nanoseconds. HTTP/gRPC call = 1-10ms in best case, 100ms+ across regions. Chain 5 services = 50ms+ latency added.

Cost 2: Partial failure

Monolith: hoặc tất cả lên, hoặc tất cả down. Distributed: service A down, B up — phải handle gracefully (circuit breaker, fallback).

Cost 3: Eventual consistency

Distributed transaction phức tạp. Đa số case dùng eventual consistency (saga, event-driven). Business team phải hiểu data có thể stale.

Cost 4: Distributed tracing

Bug trong distributed □ phải trace request qua N services. Cần Jaeger/Zipkin + correlation ID. Setup phức tạp.

Cost 5: Operational complexity

- 1 monolith: monitor 1 service, 1 deploy pipeline.
- 20 microservices: monitor 20 services, 20 pipelines, service mesh, secrets management...

Operational cost (DevOps, SRE) tăng 5-10x.

Cost 6: Testing

End-to-end test cần spin up multiple services. Slow, flaky. Cần test pyramid khác (more contract testing, less E2E).

Cost 7: Network security

Mỗi inter-service call cần TLS, auth (mTLS hoặc JWT). Cert rotation, secret management.

Trade-off table

QA	Monolithic	Distributed
Performance	High (in-process)	Medium-Low (network)
Scalability (uniform)	Vertical scale only	Horizontal scale per service
Scalability (varied)	Cannot scale parts	Excellent — scale only hot parts
Availability	All or nothing	Partial — fault isolated
Maintainability	Easy when small, hard when large	Harder per service but easier across team
Deployability	1 deploy = all changes	Independent deploys
Testability	Easy (in-process)	Hard (E2E + contract test)
Observability	Easy (1 log stream)	Hard (distributed tracing required)
Time-to-market (early)	Fast	Slow (setup overhead)
Time-to-market (mature, large team)	Slow (merge hell)	Fast (parallel team work)
Cost (small)	Low (\$50-500/mo)	High (\$500-5000/mo)
Cost (large)	High (1 huge server)	Optimized (scale only needed parts)

Insight: trade-off đảo theo scale. Monolithic tốt ở scale nhỏ, distributed tốt ở scale lớn — crossover khoảng 20-50 engineers + 100k+ users.

Modular Monolith — middle ground

Trước khi nhảy lên distributed, consider modular monolith:

- 1 deployment (1 process, 1 DB).
- *Nhưng* code organized thành strict modules với boundary cứng (mỗi module có interface công khai, không import internal lẫn nhau).
- Build tool enforce: module A không import từ `module_b.internal.*`.

Lợi ích:

- Có lợi ích của monolithic (simple deploy, in-process call).
- Có lợi ích của modularity (vertical slicing, team ownership).
- Migrate sang microservices dễ sau này (mỗi module $\square$ service).

Tools: Java module-info.java, Spring Modulith, .NET assemblies, Python package structure with strict imports.

Heuristic: hầu hết “we need microservices” thực sự “we need modular monolith”. Cố modular monolith trước, distributed khi pain thực sự.

Strangler Fig pattern

Khi đã có monolith và cần lên distributed, đừng rewrite from scratch (high risk). Dùng Strangler Fig:

1. Build API gateway/proxy in front of monolith.
2. Cứ feature mới, build microservice mới, route traffic qua gateway.
3. Dần “strangle” monolith — chuyển features hiện có thành microservice từng phần.
4. Sau N năm, monolith biến mất.

Pattern này safe, gradual, không big-bang rewrite.

Sai lầm thường gặp

Sai lầm 1: Microservices vì hype

Đã nói nhiều. Resume-driven architecture.

Sai lầm 2: Distributed monolith

Tách thành microservices mà coupling chặt. Đã nói ở Bài 3.4.

Sai lầm 3: Quên 8 fallacies

Code distributed như local. Result: bug bí ẩn trong production.

Sai lầm 4: Không có observability từ ngày 1

Build distributed mà không có distributed tracing/centralized logging. Debug trở thành mò mẫm.

Sai lầm 5: Premature optimization

“We’ll need microservices in 5 years, let’s start now.” Sai. 80% project chết trước khi cần microservices. Start monolith, migrate when needed.

Tóm tắt

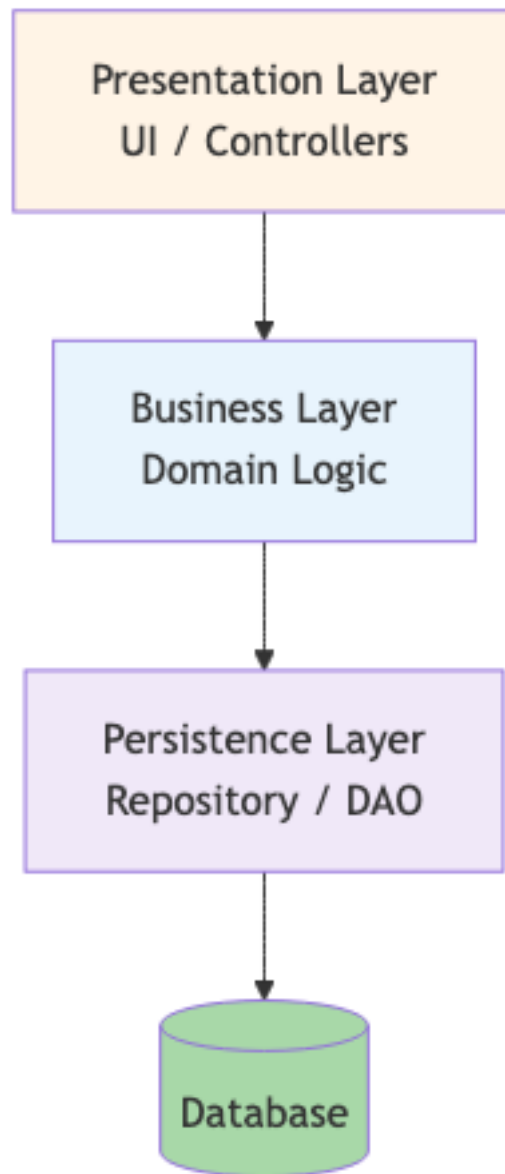
- **Monolithic default.** Distributed chỉ khi có lý do mạnh.
- **Cost của distributed:** latency, partial failure, consistency, ops, testing, security.
- **8 fallacies** của distributed computing — đọc kỹ.
- **Modular monolith** là middle ground tốt.
- **Strangler Fig** để migrate monolith → distributed an toàn.

Bài tiếp: [Layered Architecture](#) — style phổ biến nhất historical.

5.3 Layered Architecture

Tóm tắt một dòng: Hệ chia thành tầng theo technical concern (UI, Business, Data). Đơn giản, được hiểu rộng, nhưng dễ rơi vào “sinkhole anti-pattern” và khó scale theo domain.

Topology



Hình 3: Sơ đồ 12

Quy tắc cốt lõi: **dependency flow downward**. Tầng trên gọi tầng dưới, không bao giờ ngược lại.

Closed vs Open layers

Closed layer (default)

Request phải đi qua *mọi* tầng. Presentation không skip Business để gọi Persistence trực tiếp.

Pros: enforce layering, encapsulation.

Cons: boilerplate khi tầng giữa chỉ “pass-through”.

Open layer

Một layer có thể be “open” — tầng trên có thể skip nó.

Vd: introduce Shared Services layer mà Presentation có thể skip:

Sơ đồ Mermaid (không render được, xem trên web)

graph TD

```
P[Presentation]
SS[Shared Services<br/>(OPEN)]
B[Business]
PS[Persistence]

P --> B
P -.skip if simple.-> SS
B --> SS
B --> PS
```

Pros: avoid pass-through for simple ops.

Cons: phá thuần khiết. Phải document rõ layer nào open.

Heuristic: default closed. Open layer chỉ khi pass-through pain rõ.

Sinkhole Anti-pattern

Khi 80%+ requests chỉ “pass through” các layer mà không có logic:

```
# Controller
def get_user(id):
    return user_service.get_user(id)

# Service
class UserService:
    def get_user(self, id):
        return self.repo.find_by_id(id)

# Repo
class UserRepo:
    def find_by_id(self, id):
        return db.query("SELECT * FROM users WHERE id=?", id)
```

3 tầng nhưng không tầng nào thực sự *thêm logic*. Đây là sinkhole — wasted abstraction.

Fix:

- **Option A:** Allow open layers cho pass-through cases.
- **Option B:** Drop layering cho read-heavy cases, use CQRS — write qua layered, read direct query.
- **Option C:** Reconsider style. Có thể layered không fit; xem service-based hoặc microkernel.

Khi dùng Layered

□ Phù hợp khi:

- Hệ nhỏ-trung (< 50k LOC).
- Team < 15 people.
- Domain đơn giản (CRUD nặng).
- Cần onboard nhanh — layered được dạy phổ biến.
- Cần consistency với codebase legacy.

□ Không phù hợp khi:

- Hệ phức tạp với nhiều domain (vertical slicing tốt hơn).
- Cần high scalability per-domain.
- Cần tách team theo domain (Conway's law conflict).
- Performance critical với many sinkholes.

Trade-off với QA

QA	Score	Note
Simplicity	□□□□□	Cấu trúc dễ hiểu nhất
Cost	□□□□□	1 deploy, 1 DB
Testability	□□□	OK nếu mỗi layer test với mock
Modularity (technical)	□□□	Layered theo tech concern
Modularity (domain)	□	Tệ — feature đung mọi layer
Scalability	□	Cannot scale parts separately
Deployability	□□	1 deploy = mọi thay đổi
Performance	□□□□	In-process call fast
Fault tolerance	□□	Single point of failure

Layered tối ưu cho **simplicity + cost**, hy sinh **scalability + modularity domain**.

Implementation trong code

Java/Spring example

```
// presentation/UserController.java
@RestController
@RequestMapping("/users")
public class UserController {
    private final UserService service;

    @GetMapping("/{id}")
    public UserDto get(@PathVariable Long id) {
        return service.getUser(id);
    }
}

// business/UserService.java
@Service
public class UserService {
```

```

private final UserRepository repo;

public UserDto getUser(Long id) {
    User user = repo.findById(id).orElseThrow();
    return UserDto.from(user); // mapping
}

public UserDto createUser(CreateUserRequest req) {
    // validation, business rule
    User user = new User(req.email(), req.name());
    user.validate();
    return UserDto.from(repo.save(user));
}
}

// persistence/UserRepository.java
@Repository
public interface UserRepository extends JpaRepository<User, Long> {}

```

Mỗi layer in own package. Spring auto-wire dependencies.

Python/Flask example

```

# presentation/user_blueprint.py
@app.route("/users/<int:id>")
def get_user(id):
    return jsonify(user_service.get(id))

# business/user_service.py
class UserService:
    def __init__(self, repo): self._repo = repo
    def get(self, id): return UserDto.from_entity(self._repo.find(id))

# persistence/user_repository.py
class UserRepository:
    def find(self, id):
        return User.query.filter_by(id=id).first()

```

Variants

Layered + Hexagonal

Combine layered với hexagonal (Cụm 2.7 DIP). Business layer định nghĩa abstraction; Persistence layer implement. Decouple business khỏi infrastructure.

Onion / Clean Architecture

Layered hơi twist: domain ở center, infrastructure ở ngoài. Dependency direction *vào trong* (DIP applied). Effectively layered with DIP enforced.

Cụm này không đi sâu — đọc *Clean Architecture* (Robert Martin) sau khoá.

Sai lầm thường gặp

Sai lầm 1: Skip layers ad-hoc

Code nhanh, controller gọi thẳng repository. Vài tháng sau, business layer chỉ là half-empty shell — không reliable để add logic.

Fix: discipline. Default closed; document open layer rõ ràng.

Sai lầm 2: Layered cho complex domain

Hệ có 20 sub-domain. Layered chỉ có 3 tầng — không match domain complexity. Mỗi feature touch nhiều file.

Fix: vertical slice (theo domain) + layered bên trong mỗi domain.

Sai lầm 3: Database-first design

Bắt đầu thiết kế từ database schema, work upward. Result: business layer là CRUD wrapper, không thực sự reflect domain.

Fix: design domain model first (business layer), DB schema follow. Hexagonal/Clean approach.

Sai lầm 4: Repository pattern abuse

Repository làm everything: SQL, caching, validation, business logic. Vi phạm SRP.

Fix: repository chỉ persistence. Cache trong service layer. Validation trong domain.

Tóm tắt

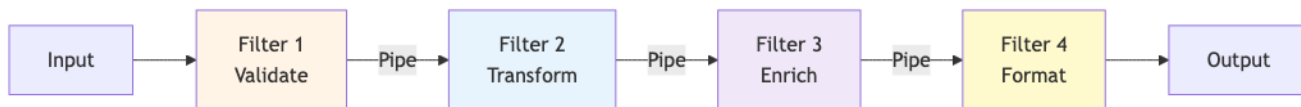
- **Layered**: tầng UI / Business / Persistence theo technical concern.
- **Closed default**, open chỉ khi pain rõ.
- **Sinkhole** = anti-pattern: pass-through layers.
- **Phù hợp**: hệ nhỏ-trung, domain đơn giản, team < 15.
- **Trade-off**: simplicity + cost tối ưu; scalability + domain modularity hy sinh.

Bài tiếp: [Pipeline Architecture](#) — sequential transformation, Unix philosophy.

5.4 Pipeline Architecture

Tóm tắt một dòng: Hệ thống như một dây chuyền lắp ráp: data đi vào filter đầu tiên, được transform, đi qua filter tiếp, và cứ thế. Style này tối ưu cho composability và là backbone của Unix tools, compiler, ETL pipeline.

Topology cốt lõi



Hình 4: Sơ đồ 14

Hai thành phần:

- **Filter:** đơn vị xử lý. Nhận input, biến đổi, xuất output. Self-contained, không biết về filter trước hay sau.
- **Pipe:** kênh chuyển data giữa filter. Có thể là in-memory queue, file, network stream.

Quy tắc cốt lõi: **filter không lưu state giữa các message**. Mỗi message processed độc lập (stateless filter).

Bốn loại filter

1. Producer (source)

Generate data từ nguồn ngoài (file, DB, API). Không có input. Vd: `cat file.txt` đọc file vào pipeline.

2. Transformer

Map 1 input → 1 output. Vd: `grep "ERROR" logs.txt`.

3. Tester

Filter input theo predicate. Vd: chỉ pass record có `status="success"`.

4. Consumer (sink)

Ghi output đến đích cuối. Không có output downstream. Vd: `> output.txt`.

Unix Philosophy

Pipeline architecture là cụ thể hoá Unix Philosophy của Doug McIlroy:

“Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.”

Vd Unix pipeline:

```
cat access.log | grep "404" | awk '{print $7}' | sort | uniq -c | sort -rn | head -10
```

Pipeline này:

1. `cat access.log` — producer.

2. `grep "404"` — tester (only 404 errors).
3. `awk '{print $7}'` — transformer (extract URL).
4. `sort` — transformer (sort).
5. `uniq -c` — transformer (count duplicates).
6. `sort -rn` — transformer (sort by count desc).
7. `head -10` — tester (top 10 only).

Mỗi command đơn giản. Compose thành workflow mạnh. SRP và OCP scaled lên hệ.

Khi dùng Pipeline

☐ Phù hợp:

- **Data processing pipeline:** ETL (Extract-Transform-Load), data warehouse, ML feature pipeline.
- **Compiler:** source ☐ tokenize ☐ parse ☐ AST ☐ optimize ☐ codegen.
- **Image/video processing:** load ☐ resize ☐ filter ☐ encode.
- **Log/event processing:** ingest ☐ parse ☐ enrich ☐ store/route.
- **Build system:** compile ☐ link ☐ package ☐ deploy.

☐ Không phù hợp:

- Workflow có business logic phức tạp (loop, branch, retry).
- Cần state shared giữa các step.
- User-facing interactive system.

Cách implement

Synchronous (in-process)

```
def pipeline(data):
    return format_output(
        enrich(
            transform(
                validate(data)
            )
        )
    )

# Hoặc functional
from functools import reduce
filters = [validate, transform, enrich, format_output]
result = reduce(lambda d, f: f(d), filters, input_data)
```

Đơn giản. Mọi filter chạy trong cùng process.

Async với queue (in-process)

```
import queue
import threading

q1 = queue.Queue()
q2 = queue.Queue()
```

```

q3 = queue.Queue()

def filter1():
    for item in source:
        q1.put(transform1(item))

def filter2():
    while True:
        item = q1.get()
        q2.put(transform2(item))

# ... spawn thread mỗi filter

```

Mỗi filter chạy thread riêng. Throughput cao hơn (parallel work).

Distributed (network)

Stream processing framework: Kafka Streams, Apache Flink, Spark Streaming.

```

Source (Kafka topic A)
  → Filter 1 (Flink job)
    → Topic B
      → Filter 2 (Flink job)
        → Topic C
          → Sink (Kafka topic D)

```

Mỗi filter là một service/job. Pipe = message topic. Scale horizontally.

Trade-off với QA

QA	Score	Note
Composability	□□□□□	Add filter mới rất dễ
Reusability	□□□□□	Filter dùng được ở nhiều pipeline
Modularity	□□□□	Mỗi filter SRP cao
Performance (throughput)	□□□□	Pipeline parallel tốt
Performance (latency)	□□□	Latency = sum của filter latencies
Testability	□□□□□	Mỗi filter test isolated
Maintainability	□□□□	Thêm/đổi filter localized
Complexity (stateful workflow)	□□	Tệ khi cần state hoặc branch

Pipeline tối ưu cho **composability + reusability + testability**.

Ví dụ thực tế

Compiler (GCC, LLVM)

```

Source code → Lexer → Parser → AST → Type Checker →
  IR Generator → Optimizer (multiple passes) →
  Code Generator → Assembler → Linker → Binary

```

Mỗi pass là một filter. Output của filter này là input cho filter sau. LLVM nổi tiếng vì design pipeline cực kỳ modular — viết optimizer mới = drop in một filter.

ETL Pipeline (Airflow, Luigi)

```
# Airflow DAG (pseudo-code)
extract_users = PythonOperator(task_id='extract', python_callable=extract_from_db)
clean_users = PythonOperator(task_id='clean', python_callable=remove_pii)
enrich_users = PythonOperator(task_id='enrich', python_callable=join_with_segments)
load_users = PythonOperator(task_id='load', python_callable=load_to_warehouse)

extract_users >> clean_users >> enrich_users >> load_users
```

Mỗi task là một filter. Airflow schedule và monitor.

Image Processing (PIL, OpenCV)

```
from PIL import Image

img = Image.open("input.jpg")
img = img.resize((800, 600))
img = img.convert("L") # grayscale
img = img.filter(ImageFilter.BLUR)
img.save("output.png")
```

Mỗi method là một filter. Chain qua method call.

CI/CD Pipeline

```
git push → Trigger CI →
  Filter 1: Install dependencies
  Filter 2: Lint
  Filter 3: Unit test
  Filter 4: Build artifact
  Filter 5: Security scan
  Filter 6: Deploy staging
  Filter 7: Smoke test
  Filter 8: Deploy prod
```

Mỗi step có thể fail □ pipeline halt. Pure pipeline architecture.

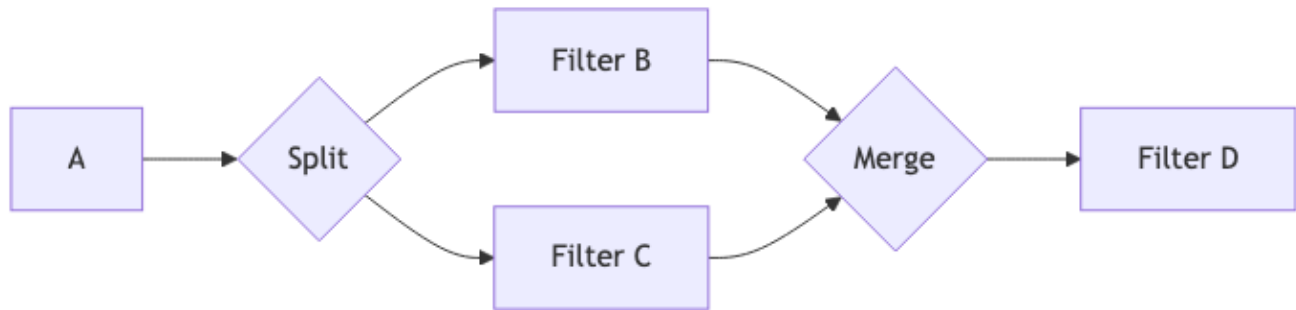
Variants

Linear pipeline

Filter A □ B □ C □ D. Đơn giản nhất.

Branching pipeline

Filter A □ split into B and C parallel □ merge into D.



Hình 5: Sơ đồ 15

Use case: validate cho cả format A và format B song song, merge result.

DAG pipeline (Directed Acyclic Graph)

Pipeline phức tạp với multiple producers, transformers, consumers, dependencies giữa nhau.

Airflow là example. Build system (Make, Bazel) cũng là DAG pipeline.

Sai lầm thường gặp

Sai lầm 1: Stateful filter

Filter share state qua global variable hoặc DB. Phá lợi ích chính của pipeline (parallel, replay, debug).

Fix: stateless filter. State chỉ ở pipe (queue/topic) hoặc external store (Redis, DB).

Sai lầm 2: Tightly coupled filters

Filter A biết Filter B sẽ nhận output $\square$ format theo expectation của B. Phá composability.

Fix: filter chỉ produce data theo well-defined schema. Filter sau adapt nếu cần.

Sai lầm 3: God filter

Filter làm quá nhiều việc (validate + transform + enrich + log + alert). Vi phạm SRP.

Fix: tách thành nhiều filter nhỏ. Pipeline architecture *expect* nhiều filter.

Sai lầm 4: No error handling

Filter throw $\square$ pipeline halt cả batch. Phải có dead-letter queue / error sink.

Fix: mỗi filter có error handling. Failed message route vào dead-letter pipe cho retry/inspect.

Sai lầm 5: Pipeline cho non-pipeline workload

User registration flow: validate input $\square$ check duplicate $\square$ save DB $\square$ send welcome email. *Có thể* model như pipeline nhưng:

- Step 3 (save DB) có business logic phức tạp (transaction, rollback).

- Step 4 (email) là side effect, không phải data transformation.

Style này phù hợp với layered hoặc service-based hơn. Pipeline phù hợp với pure data flow.

Tóm tắt

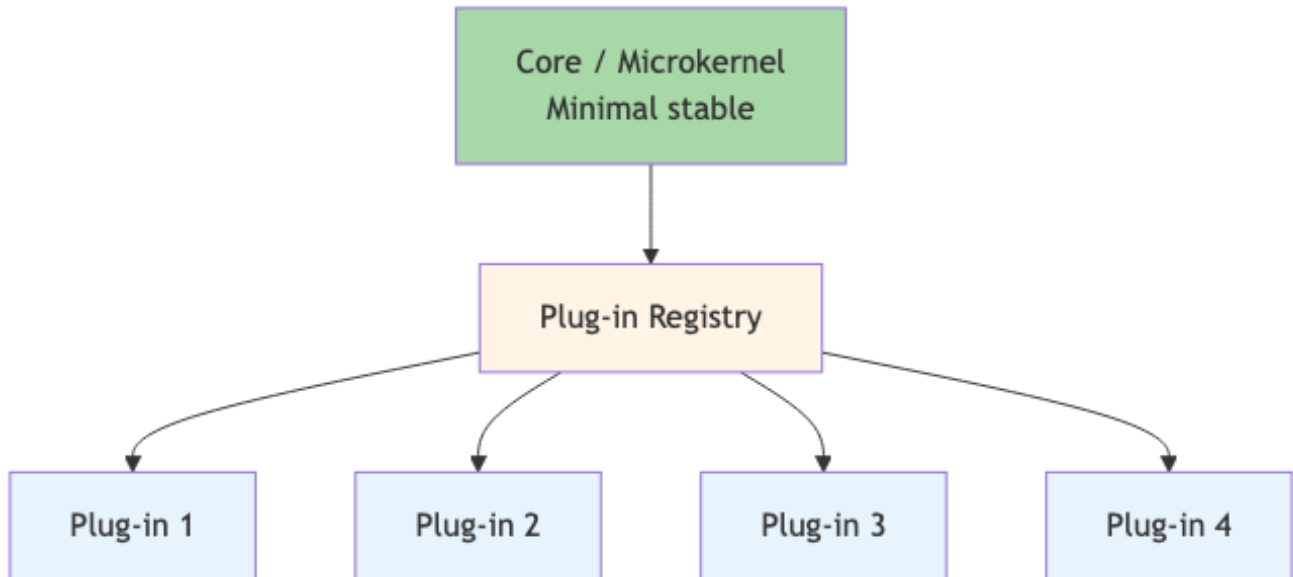
- **Pipeline**: filter (stateless) + pipe (channel).
- **4 loại filter**: producer, transformer, tester, consumer.
- **Unix philosophy**: do one thing well, work together via text stream.
- **Use cases**: ETL, compiler, image processing, CI/CD, log processing.
- **Trade-off**: composability + reusability tối ưu; phức tạp khi cần state hoặc branch logic.

Bài tiếp: [Microkernel Architecture](#) — OCP ở scale lớn, core + plug-in.

5.5 Microkernel Architecture

Tóm tắt một dòng: Hệ thống chia thành một core stable nhỏ + nhiều plug-in mở rộng tính năng. OCP ở scale lớn. Phù hợp cho product cần third-party extension hoặc customization mạnh.

Topology cốt lõi



Hình 6: Sơ đồ 16

Ba thành phần:

- **Core (kernel):** minimal, stable. Chứa logic *chung nhất* (vd: window management, plug-in lifecycle, common utilities).
- **Plug-in:** standalone module. Implement một feature cụ thể. Communicate với core qua well-defined contract.
- **Registry:** nơi core biết về plug-in nào available. Discovery cơ chế: file system scan, manifest config, hoặc remote registry.

Quy tắc cốt lõi: **core không biết về plug-in cụ thể**. Plug-in implement interface mà core defined. Add/remove plug-in không touch core code.

Lợi ích vs Layered

So với layered:

- **Extensibility:** layered cần modify code để add feature. Microkernel chỉ cần drop in plug-in.
- **Customization:** user/admin có thể enable/disable plug-in. Layered có toàn bộ feature on-or-off.
- **Third-party ecosystem:** plug-in API mở cho dev khác. Eg: VS Code extension marketplace.
- **Independent versioning:** core và plug-in version độc lập.

Trade-off với layered:

- **Complexity:** phải design plug-in API cẩn thận, manage lifecycle (load, init, dispose).
- **Performance:** cross-plug-in call có overhead (interface dispatch, marshal data).
- **Security:** plug-in third-party có thể malicious — cần sandboxing.

Examples thực tế

VS Code

- **Core:** window manager, file system access, editor core, plug-in lifecycle.
- **Plug-in:** hàng ngàn extension cho language, theme, debugging, AI assistance, source control.
- **API:** TypeScript SDK với contracts (commands, providers, language services).

Eclipse IDE

Tiền bối của VS Code. Core (Equinox OSGi container) + bundles (plug-ins). Mọi feature kể cả Java editor là plug-in.

Chrome / Firefox

- **Core:** Chromium / Gecko engine.
- **Plug-in:** extension cho ad-blocker, password manager, dev tools.

WordPress

- **Core:** post system, user management, admin UI, plug-in lifecycle.
- **Plug-in:** SEO, e-commerce (WooCommerce), gallery, forms, ... > 50,000 plug-in.

CMS / E-commerce platforms

Magento, Shopify, Drupal đều microkernel. Merchant install plug-in để add feature mà không cần touch core.

DAW (Digital Audio Workstation)

Pro Tools, Ableton: core audio engine + plug-in synthesizer/effect (VST3, AU).

Plug-in Contract Design

Đây là phần khó nhất. Contract = interface mà mọi plug-in phải implement. Thiết kế kém ☐ plug-in ecosystem chết.

Loại 1: Event/Hook system

Core emit event. Plug-in subscribe.

```
# Core
class EventBus:
    def emit(self, event_name, payload):
        for handler in self._handlers.get(event_name, []):
            handler(payload)

# Plug-in
def on_user_login(payload):
    log.info(f"User {payload.user_id} logged in")
```



```
core.event_bus.subscribe("user.login", on_user_login)
```

Vd: WordPress hooks (add_action, add_filter).

Pros: loose coupling. Plug-in không block core.

Cons: hard to reason about order; debugging khó.

Loại 2: Service registration

Plug-in register service vào registry. Core query.

```
# Plug-in
class MyAuthProvider(AuthProvider):
    def authenticate(self, creds): ...

core.registry.register("auth", "oauth", MyAuthProvider())

# Core
auth = core.registry.get("auth", "oauth")
user = auth.authenticate(creds)
```

Vd: Spring beans, Java ServiceLoader, .NET DI container.

Pros: explicit, type-safe.

Cons: plug-in phải khai báo trước khi dùng.

Loại 3: Command pattern

Core có command registry. Plug-in register commands.

```
// VS Code extension
context.subscriptions.push(
    vscode.commands.registerCommand('myExt.helloWorld', () => {
        vscode.window.showInformationMessage('Hello!');
    })
);
```

Pros: discoverability (user thấy commands trong palette).

Cons: chỉ cho command-style interaction, không cho data flow phức tạp.

Loại 4: Contribution points (Eclipse model)

Core declare contribution point. Plug-in declare what it contributes.

```
<!-- Plug-in's plugin.xml -->
<extension point="org.eclipse.ui.editors">
    <editor id="my.editor" class="my.EditorImpl"/>
</extension>
```

Core scan all plug-ins for contributions, wire up.

Pros: declarative, lazy loading.

Cons: XML configuration verbose; learning curve cao.

Plug-in Lifecycle

Plug-in cần được manage:

1. **Discovery**: core scan plug-in folder, registry, hoặc remote source.
2. **Load**: load code (DLL, JAR, JS bundle) vào process.
3. **Initialize**: gọi plug-in's `init()`; plug-in register services/commands.
4. **Activate**: plug-in start using core services. Lazy activation = activate khi user thật sự dùng.
5. **Deactivate**: clean up resources, unsubscribe events.
6. **Unload**: remove code khỏi memory.

Mỗi step có error handling. Bad plug-in không được crash core.

Sandboxing (cho third-party plug-ins)

Plug-in third-party có thể malicious hoặc bug. Cần isolation:

Process isolation

Plug-in run trong child process. Crash plug-in không vỡ core. IPC qua pipe/socket.

Vd: Chrome render extension trong child process.

Cost: IPC slow hơn in-process call (~100x).

Sandboxed runtime

Plug-in run trong VM với limited permission. Vd: V8 isolate, Wasm sandbox.

Cost: limited capability (no syscalls, limited memory).

Permission system

Plug-in declare permission cần (network, file system, mic, camera). User approve khi install.

Vd: Browser extension permissions, Android app permissions.

Trade-off với QA

QA	Score	Note
Extensibility	□□□□□	Best in class
Modularity	□□□□	Core + plug-in boundary cứng
Performance	□□□	Plug-in call có overhead
Maintainability (core)	□□□□	Core nhỏ, stable, dễ maintain
Maintainability (plug-in)	□□	Plug-in compatibility issue khi core update
Testability	□□□	Test plug-in dễ; test core+plug-in interactions khó
Deployability	□□□□	Plug-in deploy/update độc lập

QA	Score	Note
Complexity	□□	Plug-in API design + lifecycle phức tạp

Microkernel tối ưu cho **extensibility** — sacrificing performance + simplicity.

Khi dùng Microkernel

□ Phù hợp:

- Product cần customization mạnh (CMS, e-commerce, IDE).
- Có ecosystem third-party (extension marketplace).
- Core feature ổn định, variation ở edge cases.
- Cần independent versioning core vs feature.

□ Không phù hợp:

- App nội bộ với fixed feature set.
- Performance critical (real-time, low-latency).
- Team không có resource để maintain plug-in API.

Variants

Microkernel + Distributed plug-ins

Plug-in chạy ở remote service. Core gọi qua HTTP/gRPC.

Vd: cloud services with extension hooks (Salesforce, Zendesk).

Cost: thêm distributed cost (Bài 5.2).

Microkernel + Module Federation (Frontend)

Webpack 5 Module Federation: frontend microkernel với React modules từ multiple repos.

```
// host (microkernel)
import('plugin/Feature').then(module => {
  module.default.mount('#container');
});
```

Plug-in code loaded lazily từ CDN. Independent deployment cho frontend teams.

Sai lầm thường gặp

Sai lầm 1: Core quá lớn

Core “nhỏ” theo plan nhưng dần phình ra với utilities, common features. Plug-in API không thực sự minimal.

Fix: discipline. Mọi feature mới hỏi “đây có phải core không?”. Default: not core, làm plug-in.

Sai lầm 2: API không backward compatible

Core release mới phá plug-in cũ. Ecosystem chết.

Fix: SemVer cho plug-in API. Major version chỉ khi *force break*. Deprecate API trước khi remove.

Sai lầm 3: Plug-in coupling lẫn nhau

Plug-in A depend Plug-in B. Khi cài A không có B ÷ vỡ.

Fix: plug-in chỉ depend Core API. Coordination giữa plug-in qua event bus của core, không direct call.

Sai lầm 4: Performance overhead bị bỏ qua

Plug-in call qua reflection/dispatcher. Slow trong hot path.

Fix: profile hot path. Cache plug-in instance. Pre-compile plug-in dispatch.

Sai lầm 5: Security model yếu

Plug-in có full access vào core internal. Một bad plug-in crash hoặc data theft.

Fix: define permission model. Sandbox plug-in. Code-review marketplace.

Tóm tắt

- **Microkernel**: core stable + plug-ins extensible.
- **OCP ở scale lớn**: core closed for modification, open for extension via plug-in.
- **Plug-in contract**: event/hook, service registration, command, contribution point.
- **Lifecycle + sandboxing** quan trọng cho production.
- **Use cases**: IDE, browser, CMS, e-commerce, DAW.
- **Trade-off**: extensibility tối ưu, sacrificing performance + complexity.

Tổng kết Cụm 5

Hết Cụm 5. Tóm tắt 3 styles:

Style	Tối ưu	Sacrificing	Use case điển hình
Layered	Simplicity, cost	Scalability, domain modularity	CRUD app, internal tools
Pipeline	Composability, reusability	Stateful logic, branching	ETL, compiler, image processing
Microkernel	Extensibility	Performance, complexity	IDE, CMS, browser

Plus **Monolithic vs Distributed** baseline (Bài 5.2).

Cụm tiếp: [Cụm 6 - Distributed Styles](#) — bốn style phân tán quan trọng nhất.